RB-Y1 Dev

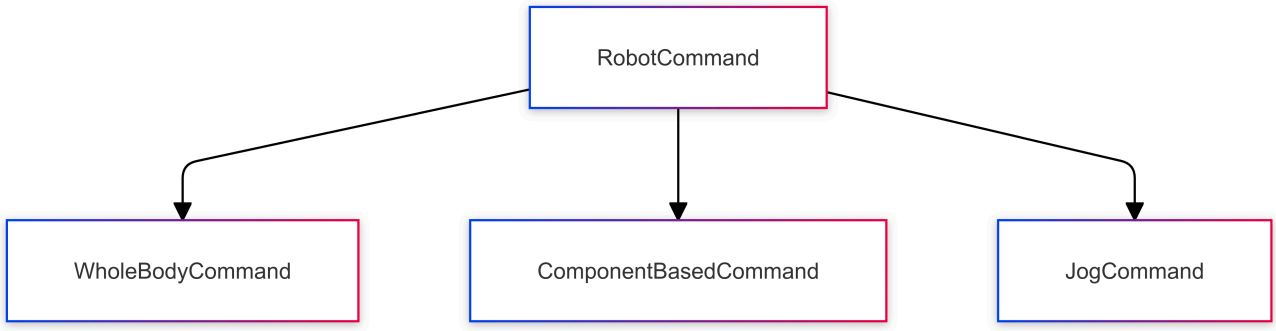
Menu On this page

Command and Control Architecture

Overview

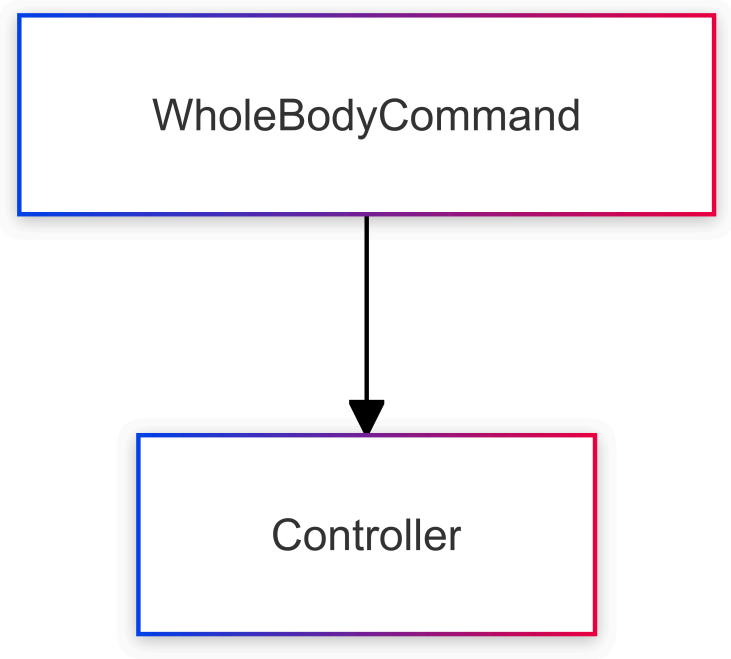
The robot commands are divided into three main types:

1. **WholeBodyCommand**: Controls the entire robot with 24 degrees of freedom (DOF) in a single command.
2. **ComponentBasedCommand**: Sends commands to the robot's `head`, `body`, or `mobility` components individually.
3. **JogCommand**: Controls smaller, individual movements, typically used for fine adjustments.



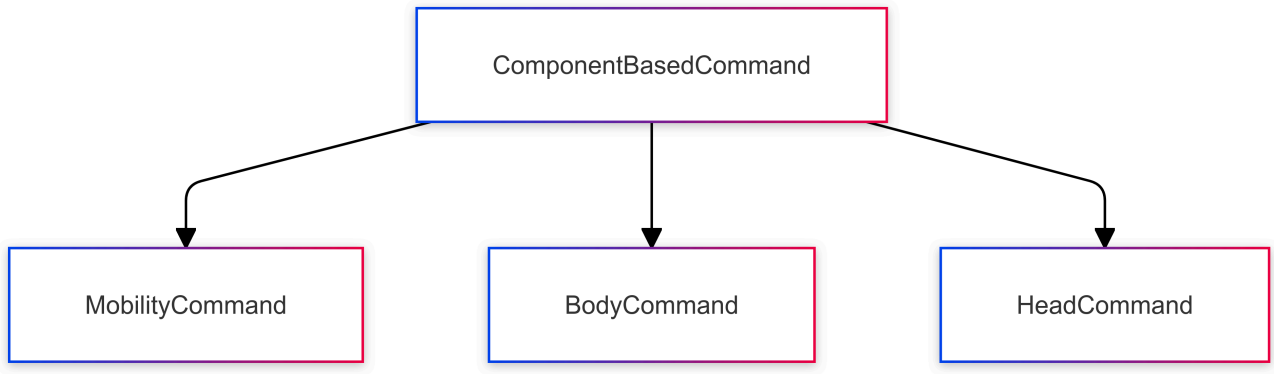
WholeBodyCommand

This command controls all the major components of the robot, including arms, torso, and legs. It supports various controllers for managing the full body of the robot. The primary use case is for synchronized movements where multiple parts of the robot are involved. This command sends a **24 DOF** control signal all at once.



ComponentBasedCommand

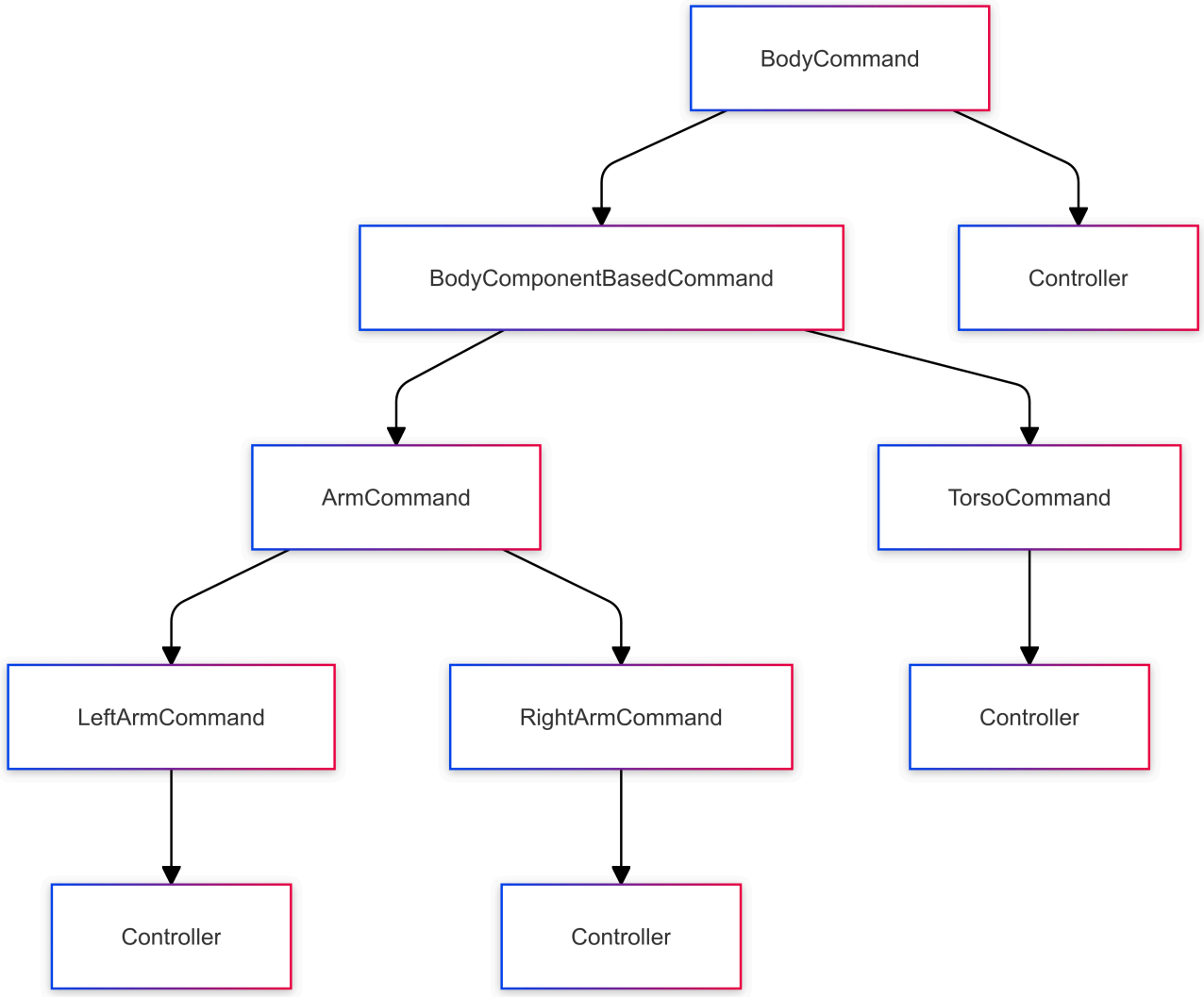
The **ComponentBasedCommand** is responsible for controlling the core components of the robot, such as the **Mobility**, **Body**, and **Head**. It provides a flexible architecture for handling different sections of the robot individually. Each of these components can be controlled using specialized commands, allowing precise control over the robot's movement and actions. This structure typically supports multiple controllers to handle specific parts of the robot efficiently.



BodyCommand Structure

The **BodyCommand** allows for detailed control of the robot's body and arms. It is divided into **BodyComponentBasedCommand**, under which **TorsoCommand** and **ArmCommand** exist. The arms can be controlled individually through **LeftArmCommand** and **RightArmCommand**, providing finer control over each arm.

In the diagram below, the **Controller** nodes follow the structure defined in the Controllers section, where each **Controller** node represents the available controllers for joint position, Cartesian motion, impedance control, gravity compensation, and optimal control.



Controllers

The following controllers are used throughout different commands like **WholeBodyCommand**, **ComponentBasedCommand**, and their subcommands.

- **JointPositionController**: Manages the position of each joint.
- **CartesianController**: Controls the Cartesian position of the robot.
- **ImpedanceController**: Allows control of force and motion, useful for interactions with the environment.
- **GravityCompensationController**: Compensates for gravitational forces, making the robot more energy-efficient during holding positions.
- **OptimalController**: Controls the robot's motion using an optimization equation.

$$\min_{\dot{\mathbf{q}}} \left(\sum_{i=0}^{n-1} \left(W_{1,i} \left(J_{i,b} \dot{\mathbf{q}} - \log \left(T_{i,\text{cur}}^{-1} \cdot T_{i,\text{ref}} \right) \right) \right) + W_2 \left(\dot{\mathbf{q}} - (\mathbf{q}_{\text{ref}} - \mathbf{q}_{\text{cur}}) \right) + W_3 \dot{\mathbf{q}} \right)$$

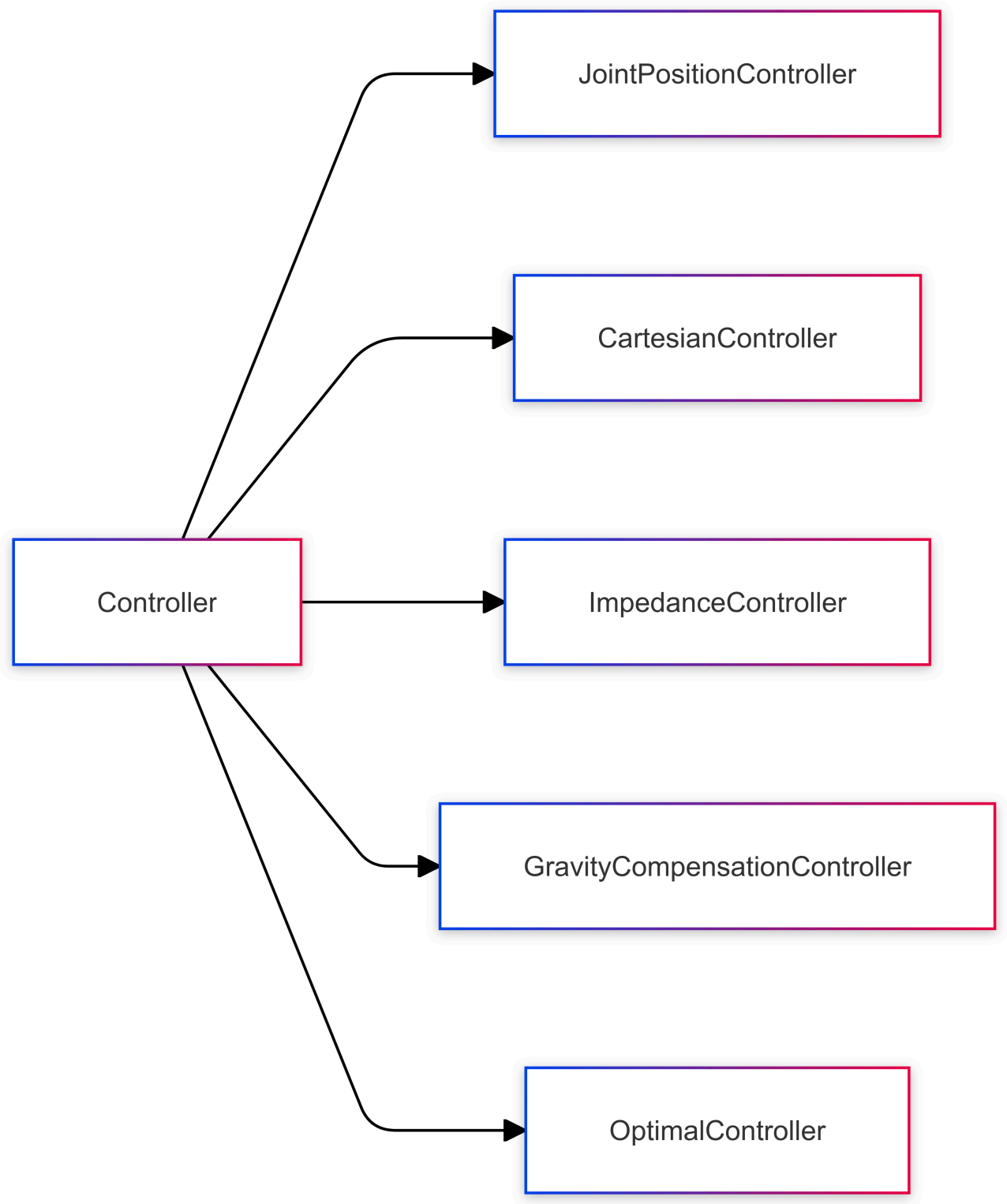
$$\dot{\mathbf{q}}_{\text{lb}} \leq \dot{\mathbf{q}} \leq \dot{\mathbf{q}}_{\text{ub}}$$

$$\mathbf{q}_{\text{lb}} - \mathbf{q}_{\text{cur}} \leq \dot{\mathbf{q}} dt \leq \mathbf{q}_{\text{ub}} - \mathbf{q}_{\text{cur}}$$

$$W_{1,i} = \text{diag}(w_{o,i}, w_{o,i}, w_{o,i}, w_{p,i}, w_{p,i}, w_{p,i})$$

$$W_2 = \text{diag}(w_0, w_1, \cdots, w_{\text{dof}-1})$$

$$W_3 = \text{constant small value}$$



Examples

python

```python
def example_joint_position_command_1(robot):
    print("joint position command example 1")

    # Initialize joint positions
```

```
        q_joint_waist = np.zeros(6)
        q_joint_right_arm = np.zeros(7)
        q_joint_left_arm = np.zeros(7)

        # Set specific joint positions
        q_joint_right_arm[1] = -90 * D2R
        q_joint_left_arm[1] = 90 * D2R

        rc = RobotCommandBuilder().set_command(
            ComponentBasedCommandBuilder().set_body_command(
                BodyComponentBasedCommandBuilder()
                .set_torso_command(
                    JointPositionCommandBuilder()
                    .set_minimum_time(minimum_time)
                    .set_position(q_joint_waist)
                )
                .set_right_arm_command(
                    JointPositionCommandBuilder()
                    .set_minimum_time(minimum_time)
                    .set_position(q_joint_right_arm)
                )
                .set_left_arm_command(
                    JointPositionCommandBuilder()
                    .set_minimum_time(minimum_time)
                    .set_position(q_joint_left_arm)
                )
            )
        )

        rv = robot.send_command(rc, 10).get()

        if rv.finish_code != RobotCommandFeedback.FinishCode.Ok:
            print("Error: Failed to conduct demo motion.")
            return 1

        return 0


    def example_joint_position_command_2(robot):
        print("joint position command example 2")

        # Define joint positions
        q_joint_waist = np.array([0, 30, -60, 30, 0, 0]) * D2R
        q_joint_right_arm = np.array([-45, -30, 0, -90, 0, 45, 0]) * D2R
```

```
    q_joint_left_arm = np.array([-45, 30, 0, -90, 0, 45, 0]) * D2R

    # Combine joint positions
    q = np.concatenate([q_joint_waist, q_joint_right_arm, q_joint_left_arm])

    # Build command
    rc = RobotCommandBuilder().set_command(
        ComponentBasedCommandBuilder().set_body_command(
            BodyCommandBuilder().set_command(
                JointPositionCommandBuilder()
                .set_position(q)
                .set_minimum_time(minimum_time)
            )
        )
    )

    rv = robot.send_command(rc, 10).get()

    if rv.finish_code != RobotCommandFeedback.FinishCode.Ok:
        print("Error: Failed to conduct demo motion.")
        return 1

    return 0
```

Previous page
Before v0.2.0

Next page
Control Hold Time and Minimum Time